

Problem set #11: Binomial Monte Carlo

Milica Cudina

2026-07-14

Let the **volatility** of a stock be the standard deviation of its (continuously compounded) realized return on an annual basis. Then, we can define the up and down factors in the so-called *forward binomial tree* for a **non-dividend-paying** stock as

$$\begin{aligned} u &= e^{rh+\sigma\sqrt{h}} \\ d &= e^{rh-\sigma\sqrt{h}} \end{aligned} \tag{1}$$

Let the continuously compounded, risk-free interest rate be 0.04.

```
r=0.04
```

Consider a stock whose current price is \$100 and whose volatility is 0.25. We will be pricing a one-year, at-the-money call option in a variety of ways here.

```
#about the stock
```

```
s0=100
```

```
sigma=0.25
```

```
#about the call
```

```
T=1
```

```
K=s0
```

Problem #1: Analytic one period

Price the option above using a one period binomial tree.

```
# number of periods
```

```
n=1
```

```
# length of one period
```

```
h=T/n
```

```
# define the factors in our forward binomial tree
```

```
(u=exp(r*h+sigma*sqrt(h)))
```

```
## [1] 1.336427
```

```
(d=exp(r*h-sigma*sqrt(h)))
```

```
## [1] 0.8105842
```

```
# the risk-neutral probability
```

```
(p=(exp(r*h)-d)/(u-d))
```

```
## [1] 0.4378235
```

```
# the forward-tree shortcut
(p.tilde=1/(1+exp(sigma*sqrt(h))))
```

```
## [1] 0.4378235
```

```
# the possible stock prices
(s.u=u*s0)
```

```
## [1] 133.6427
```

```
(s.d=d*s0)
```

```
## [1] 81.05842
```

```
# the possible payoffs
(v.u=max(s.u-K,0))
```

```
## [1] 33.64275
```

```
(v.d=max(s.d-K,0))
```

```
## [1] 0
```

```
# the initial option price
(v.0=exp(-r*T)*(v.u*p+v.d*(1-p)))
```

```
## [1] 14.15203
```

Problem #2: Monte Carlo one period

Price the option above using Monte Carlo a one period binomial tree. Use 10000 simulations.

```
# the number of simulations
n.sim=10^5
```

```
# possible results of a cointoss
factors=c(u,d)
# their risk-neutral probabilities
probs=c(p, 1-p)
```

```
# simulated stock prices
s.T=s0*sample(factors, size=n.sim, prob=probs, replace=TRUE)
```

```
# simulated payoffs
v.T=pmax(s.T-K,0) # caveat: it MUST be the pointwise max
v.T
```

```
## [1] 0.00000 0.00000 33.64275 0.00000 33.64275 0.00000 33.64275 0.00000
## [9] 33.64275 33.64275 33.64275 33.64275 0.00000 0.00000 0.00000 0.00000
## [17] 0.00000 0.00000 0.00000 0.00000 33.64275 33.64275 33.64275 33.64275
## [25] 33.64275 0.00000 0.00000 0.00000 0.00000 0.00000 0.00000 0.00000
## [33] 33.64275 33.64275 0.00000 33.64275 0.00000 0.00000 0.00000 33.64275
## [41] 0.00000 0.00000 0.00000 33.64275 33.64275 0.00000 33.64275 0.00000
## [49] 0.00000 0.00000 33.64275 0.00000 0.00000 33.64275 33.64275 0.00000
## [57] 0.00000 33.64275 0.00000 0.00000 0.00000 33.64275 33.64275 33.64275
## [65] 33.64275 0.00000 0.00000 0.00000 33.64275 0.00000 0.00000 0.00000
## [73] 0.00000 33.64275 33.64275 33.64275 0.00000 0.00000 0.00000 0.00000
## [81] 0.00000 33.64275 0.00000 33.64275 33.64275 0.00000 0.00000 33.64275
## [89] 33.64275 33.64275 0.00000 33.64275 0.00000 0.00000 33.64275 0.00000
```

```
## [97] 0.00000 0.00000 33.64275 0.00000
## [ reached 'max' / getOption("max.print") -- omitted 99900 entries ]

# average payoff
v.bar=mean(v.T)

# discounting
(v.0.mc=exp(-r*T)*v.bar)

## [1] 14.13737
```

Problem #3: Analytic two periods

Price the above option using a two-period binomial tree.

```
# number of periods
n=2

# length of one period
h=T/n

# define the factors in our forward binomial tree
(u=exp(r*h+sigma*sqrt(h)))

## [1] 1.217472
(d=exp(r*h-sigma*sqrt(h)))

## [1] 0.8548949

# the risk-neutral probability
(p=(exp(r*h)-d)/(u-d))

## [1] 0.4559206

# possible stock prices
(s.T=c(s0*u^2, s0*u*d, s0*d^2))

## [1] 148.22384 104.08108 73.08454

# possible payoffs
(v.T=pmax(s.T-K,0))

## [1] 48.223842 4.081077 0.000000

# the risk-neutral probabilities of reaching the final nodes
(probs=dbinom(c(2,1,0), size=n, prob=p))

## [1] 0.2078636 0.4961140 0.2960224

# expected risk-neutral payoff
(sum(probs*v.T))

## [1] 12.04866

# discounting
(v.0=exp(-r*T)*sum(probs*v.T))

## [1] 11.57622
```

Problem #4: Monte Carlo two periods

Price the option above using Monte Carlo a two period binomial tree. Use 10000 simulations.

```
# steppingstone
# simulating the heights of the final stock prices
x=rbinom(n.sim, n, p)

# simulated stock prices
s.T=s0*u^x*d^(n-x)

# simulated payoffs
v.T=pmax(s.T-K,0) # caveat: it MUST be the pointwise max
#v.T

# average payoff
v.bar=mean(v.T)

# discounting
(v.0.mc=exp(-r*T)*v.bar)
```

```
## [1] 11.57852
```

Problem #5: Analytic one hundred periods

Price the above option using a 100-period binomial tree.

```
# number of periods
n=100

# length of one period
h=T/n

# define the factors in our forward binomial tree
(u=exp(r*h+sigma*sqrt(h)))
```

```
## [1] 1.025725
```

```
(d=exp(r*h-sigma*sqrt(h)))
```

```
## [1] 0.9757001
```

```
# the risk-neutral probability
(p=(exp(r*h)-d)/(u-d))
```

```
## [1] 0.4937503
```

```
# the possible stock prices
k=n:0
s.T=s0*u^k*d^(n-k)

# the possible payoffs
v.T=pmax(s.T-K,0)

# their respective probabilities
probs=dbinom(k, size=n, prob=p)

# the risk-neutral expectation
```

```
sum(probs*v.T)
```

```
## [1] 12.32697
```

```
# discounting
```

```
(v.0=exp(-r*T)*sum(probs*v.T))
```

```
## [1] 11.84362
```

Problem #6: Monte Carlo with one hundred periods

Price the option above using Monte Carlo with a hundred period binomial tree. Use 10000 simulations.

```
# steppingstone
```

```
# simulating the heights of the final stock prices
```

```
x=rbinom(n.sim, n, p)
```

```
# simulated stock prices
```

```
s.T=s0*u^x*d^(n-x)
```

```
# simulated payoffs
```

```
v.T=pmax(s.T-K,0) # caveat: it MUST be the pointwise max
```

```
#v.T
```

```
# average payoff
```

```
v.bar=mean(v.T)
```

```
# discounting
```

```
(v.0.mc=exp(-r*T)*v.bar)
```

```
## [1] 11.91558
```